

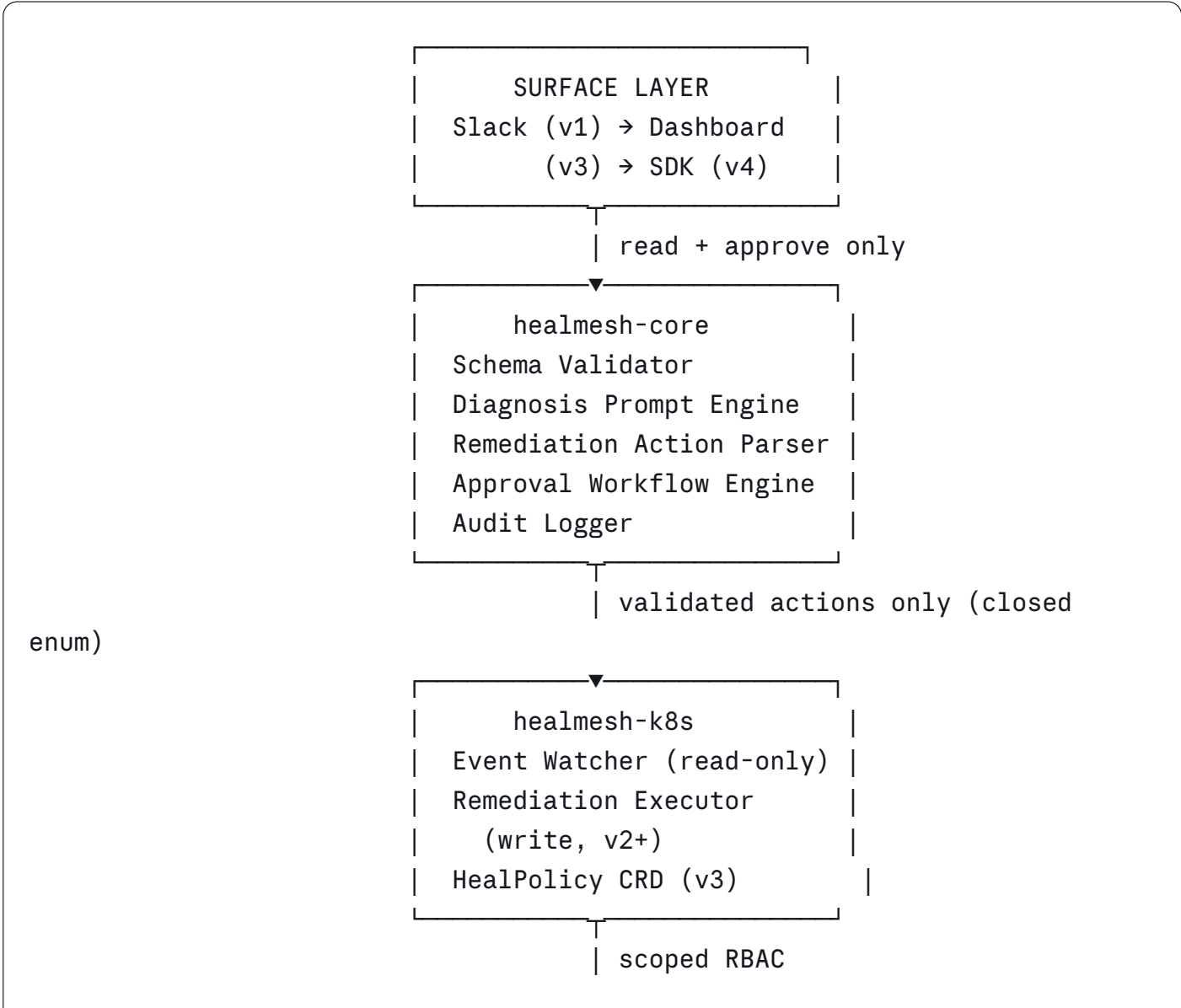
HealMesh — Architecture Reference

Status: Draft v1 Purpose: A fast, living map of the system for onboarding and day-to-day orientation. This is deliberately shorter than `docs/TDD.md` — when the two disagree, the TDD is the source of truth for implementation detail; update this file to match it, not the other way around.

1. One-paragraph summary

HealMesh watches a Kubernetes cluster, detects a fixed set of failure signatures, calls an LLM directly (no orchestration middleman) to produce a plain-language diagnosis, and — from Phase 2 onward — can execute a narrow, human-approved remediation action. The LLM's output never reaches the cluster directly: it is always parsed into a closed, typed enum first. That parsing step is the single most important architectural decision in the system.

2. System diagram



Data stores (Postgres, Vault) and cross-cutting concerns (TLS, NetworkPolicy) sit alongside this chain rather than inside it — see §5.

3. The three layers, in one line each

- **Surface layer** — how humans see diagnoses and approve actions. Never has cluster credentials. Slack in v1, Dashboard in v3, SDK in v4.
 - **healmesh-core** — the brain. Cloud-agnostic, no Kubernetes-specific code. Turns a validated incident into a diagnosis and, if approved, a validated instruction. This is where the enum contract lives.
 - **healmesh-k8s** — the hands. Watches the real cluster (read) and, from Phase 2, executes pre-approved actions (write) via a completely separate code path and identity.
-

4. The one invariant that explains most other design choices

The LLM never produces an executed command. Its output is forced into a closed enum (`PATCH`, `REDEPLOY`, `SCALE`, `HELM_UPGRADE`, `NONE`) with strictly typed parameters before anything downstream sees it. Unparseable output becomes `NONE` — diagnosis-only, no suggested automated action.

Almost every other architectural choice in this document exists to protect that one invariant:

- Direct LLM API calls (no n8n/OpenClaw) → so the parsing step lives in code the team fully owns and can audit, not inside a third-party orchestration layer.
- Read/write separation → so a compromised read-only component can't act on the cluster even if it's fully compromised.
- Hardcoded namespace denylist in code → so no config mistake can make the enum contract matter less.
- Append-only audit log at the code level → so the historical record of what the parser decided, and what a human approved, can't be quietly rewritten later.

If a future change seems to require weakening this invariant to ship a feature faster, that is a signal to redesign the feature, not the invariant (see `AGENTS.md` §1 and `CONSTITUTION.md`).

5. Data and cross-cutting layer

- **Postgres** — incidents, diagnoses, actions, approvals. Append-only via both application-level (no update/delete methods) and database-level (trigger) enforcement.
 - **Vault** (or equivalent) — all secrets: LLM API keys, Slack signing secret, Kubernetes service account tokens, DB credentials. Kubernetes Secrets are an acceptable Phase 1 stand-in per the TDD, but must be retired before any write-capable credential exists.
 - **TLS everywhere internally**, via cert-manager — set up in Phase 0, not retrofitted later.
 - **NetworkPolicy** per pod, scoped to exactly the dependencies each component needs.
 - **Dead man's switch** — external watchdog alerting if HealMesh's own heartbeat stops.
-

6. Request/data flow (Phase 2+, full flow)

1. A pod fails (CrashLoopBackOff, OOMKilled, ImagePullBackOff, failed rollout, or resource quota exceeded).
 2. Event Watcher (read-only RBAC) detects it via the Kubernetes watch API and assembles an incident payload (bounded log lines, relevant spec fields, timestamps).
 3. Incident is sent to healmesh-core over internal TLS.
 4. Schema Validator checks it against the incident contract; malformed input is rejected and logged, never passed further.
 5. Diagnosis Prompt Engine builds a structured prompt and calls the LLM API directly.
 6. Remediation Action Parser forces the LLM's response into the closed enum or `NONE`.
 7. Approval Workflow Engine routes the parsed action: today, everything requires human approval (Slack in v1, Dashboard from v3).
 8. On approval, healmesh-k8s's Remediation Executor (separate identity from the Watcher) takes a pre-action snapshot, calls the Kubernetes API directly for the scoped action, and runs a post-remediation health check.
 9. Every step above writes to the append-only Audit Logger.
-

7. Tech stack snapshot

Concern	Choice	Decided in
healmesh-core language	Python (Pydantic, direct LLM SDK)	TDD §3.2
healmesh-k8s language	Python or Go — Phase 0 decision	See <code>DECISION_LOG.md</code> once recorded
LLM provider	Evaluated in Phase 0 (Claude vs GPT-4o-class)	See <code>DECISION_LOG.md</code> once recorded
Data store	Postgres	TDD §4
Secrets	Kubernetes Secrets (Phase 1) → Vault (Phase 2+)	TDD §4
Orchestration/glue	None — direct API calls only, no n8n/OpenClaw	<code>DECISION_LOG.md</code> ADR-001

This table is a pointer, not a duplicate — check `DECISION_LOG.md` for the reasoning behind each choice and `docs/TDD.md` for full implementation detail.

8. What changed from the original prototype

The original hackathon prototype (`prism` namespace) routed diagnosis through n8n workflows and an OpenClaw AI gateway. That design is retired. See `MIGRATION.md` for exactly what carries over (Helm scaffolding, namespace/RBAC patterns, the incident JSON contract shape) and what's rebuilt from scratch (the diagnosis/parsing pipeline itself).